

AtomicSolverV3 — Hack Report

Date: 2026-04-21 **Contract:** AtomicSolverV3
(0x5B3A41826A28b6a8ed1c5F5d6052D9dB85428d59) **Exploit type:** Direct finishSolve call — unauthorized fund extraction via crafted calldata **White hat rescuer:** c0ffeebabe.eth (0xC0ffeEBABE5D496B2DDE509f9fa189C25cF29671) **Victim solver:** 0xE3FF7ccf609082BdA151F40790A1c9143B3a0663 **Amount rescued:** ~1,203,824.58 HexTrustUSD

Summary

The AtomicSolverV3 contract contained a critical vulnerability in finishSolve: any authorized address could call the function directly with crafted calldata to drain pre-approved tokens from any other authorized solver. The white hat bot c0ffeebabe.eth front-ran a potential black hat, exploited the vulnerability to rescue ~1.2M HexTrustUSD, and converted the funds via Uniswap.

Exploit Transaction Trace

Parties

Role	Address
AtomicSolverV3 (victim contract)	0x5B3A41826A28b6a8ed1c5F5d6052D9dB85428d59
Victim solver (pre-approved funds)	0xE3FF7ccf609082BdA151F40790A1c9143B3a0663
White hat MEV bot (c0ffeebabe.eth)	0xE08D97e151473A848C3d9CA3f323Cb720472D015
c0ffeebabe.eth EOA	0xC0ffeEBABE5D496B2DDE509f9fa189C25cF29671

Step-by-step

1. **MEV bot called `finishSolve` directly** — not through the AtomicQueue. Passed:

- `initiator = address(AtomicSolverV3)` — trivially bypasses the only guard
- `runData` encoding `SolveType.P2P`, `solver = 0xE3FF7...` (victim),
`minOfferReceived = 0`, `maxAssets = MAX`
- `wantApprovalAmount = 1,203,824,580,000,000,000,000,001`

2. **`_p2pSolve` executed**, producing three token events:

Log	Event	Detail
#39	<code>Transfer</code> HexTrustUSD	<code>E3FF7... → 5B3A4...</code> (AtomicSolverV3) — <code>safeTransferFrom(victim, this, amount)</code>
#40	<code>Approval</code> HexTrustUSD	<code>5B3A4...</code> approves MEV bot for same amount — <code>safeApprove(msg.sender, amount)</code> where <code>msg.sender = MEV bot</code>
#41	<code>Transfer</code> HexTrustUSD	<code>5B3A4... → MEV bot</code> — MEV bot called <code>transferFrom</code> using that approval

3. **MEV bot converted funds via Uniswap:**

- Log #42: Uniswap V4 swap — 1.2M HexTrustUSD → 54,777,912 USDC (thin pool liquidity)
 - Log #46: Uniswap V3 swap — USDC → ~0.0238 WETH
 - Log #48: WETH unwrapped to ETH
-

Root Cause

[C-1] finishSolve Callable Directly by Any Authorized Address

`finishSolve` is protected by two checks:

```
function finishSolve(...) external requiresAuth {
    if (initiator != address(this)) revert AtomicSolverV3___WrongInitiator();
    address queue = msg.sender; // attacker controls this
    ...
}
```

□

Both checks fail to provide real protection:

- `requiresAuth` — authorized bot solvers pass this check; the attacker was an authorized party
- `initiator != address(this)` — `initiator` is attacker-supplied calldata; attacker simply passes `address(AtomicSolverV3)` and the check passes trivially

There is no on-chain proof that `finishSolve` was reached via a legitimate `queue.solve()` callback. Once inside:

- `address queue = msg.sender` → `queue` = attacker's address
- `solver` decoded from `runData` → any address the attacker chooses
- `want.safeTransferFrom(victim, this, amount)` → drains victim's pre-approved tokens
- `want.safeApprove(attacker, amount)` → approves attacker to sweep

The **`AlreadyInSolveContext`** error (declared in the contract but never used) was intended to guard exactly this path. It was never wired up.

Full Audit Report

Audit Verdict: CRITICAL ISSUES — DO NOT DEPLOY

ID	Severity	Title	Status
C-1	CRITICAL	Direct <code>finishSolve</code> call drains pre-approved funds	EXPLOITED

ID	Severity	Title	Status
H-1	HIGH	Missing <code>nonReentrant</code> contradicts NatSpec — ERC777 reentrancy vector	Unpatched
H-2	HIGH	<code>safeApprove</code> without zero-reset — permanent USDT DoS	Unpatched
H-3	HIGH	Fee-on-transfer want token — approval overcommit	Unpatched
M-1	MEDIUM	CEI violation in <code>_redeemSolve</code> (bulkWithdraw before safeTransferFrom)	Unpatched
M-2	MEDIUM	Authority misconfiguration is single point of failure	Unpatched
M-3	MEDIUM	Zero-address owner not guarded in constructor	Unpatched
M-4	MEDIUM	<code>wantApprovalAmount</code> unbounded vs <code>bulkWithdraw</code> proceeds	Unpatched
L-1	LOW	Unused <code>eETH/weETH</code> constants	Unpatched
L-2	LOW	<code>AlreadyInSolveContext</code> error declared but never emitted	Unpatched
I-1	INFO	No token recovery mechanism	Unpatched

ID	Severity	Title	Status
I-2	INFO	Batch failure isolation depends on AtomicQueue	Unpatched

CRITICAL

[C-1] Direct **finishSolve** Call — Authorized Drain of Pre-Approvals

- Location: `sc.sol:98` — `finishSolve()`, `sc.sol:109` — initiator check
- **This finding was exploited.** See transaction trace above.
- Fix: Add an in-flight context lock using EIP-1153 transient storage. Set `_inSolveContext = 1` at the start of `p2pSolve/redeemSolve`, assert it in `finishSolve`, clear it after `queue.solve()` returns:

```

uint256 private transient _inSolveContext;

modifier notInSolveContext() {
    require(_inSolveContext == 0, AtomicSolverV3___AlreadyInSolveContext());
    _inSolveContext = 1;
    _;
    _inSolveContext = 0;
}

function p2pSolve(...) external requiresAuth notInSolveContext { ... }
function redeemSolve(...) external requiresAuth notInSolveContext { ... }

function finishSolve(...) external requiresAuth {
    // Replace the initiator check with the context lock assertion:
    require(_inSolveContext == 1, AtomicSolverV3___WrongInitiator());
    ...
}

```

HIGH

[H-1] Missing **nonReentrant** — NatSpec Claims Protection That Does Not Exist

- Location: `sc.sol:97` (false comment), `sc.sol:49` (`p2pSolve`), `sc.sol:70` (`redeemSolve`)

- The NatSpec on `finishSolve` states: *"nonReentrant is not needed because the above solve functions have the nonReentrant modifier."* Neither `p2pSolve` nor `redeemSolve` has `nonReentrant` — only `requiresAuth`.
- With ERC777 offer/want tokens, `safeTransfer/safeTransferFrom` callbacks allow a re-entering solver to trigger a second solve before the first completes, violating INV-2 (approval integrity).
- Fix: Add `nonReentrant` (Solmate `ReentrancyGuard`) to `p2pSolve` and `redeemSolve`. Remove the false NatSpec claim.

[H-2] `safeApprove` Without Zero-Reset — Permanent USDT DoS

- Location: `sc.sol:157` (`_p2pSolve`), `sc.sol:191` (`_redeemSolve`)
- Solmate's `safeApprove` calls `token.approve()` directly without first resetting to 0. USDT reverts when `approve(spender, X)` is called with `X > 0` and `allowance > 0`. Any partial queue fill leaves residual allowance; subsequent USDT solves revert permanently. No escape hatch exists in the contract.
- Fix: Add `want.safeApprove(queue, 0)` before both `want.safeApprove(queue, wantApprovalAmount)` calls.

[H-3] Fee-on-Transfer want Token — Approval Overcommits Contract Balance

- Location: `sc.sol:151+157` (`_p2pSolve`), `sc.sol:188+191` (`_redeemSolve`)
- `safeTransferFrom(solver, this, wantApprovalAmount)` receives `amount - fee` but `safeApprove(queue, wantApprovalAmount)` approves the gross amount. Queue `transferFrom` will attempt to pull more than the contract holds, reverting for the last user(s) in the batch.
- Fix: Reconcile actual received balance: `uint256 actual = want.balanceOf(address(this)); want.safeApprove(queue, actual);`

MEDIUM

[M-1] CEI Violation in `_redeemSolve`

- Location: `sc.sol:185` (`teller.bulkWithdraw`) before `sc.sol:188` (`safeTransferFrom`)
- External call to teller precedes the `safeTransferFrom` pull. Mitigated by trusted teller in current deployment; architectural risk if teller is ever upgraded.
- Fix: Reorder to pull `want` from solver before calling `teller.bulkWithdraw`.

[M-2] Authority Misconfiguration Collapses All Role Separation

- The contract provides no defense-in-depth — security of `finishSolve` relies entirely on the Authority contract granting that selector only to the queue. A wildcard role collapses all protection.
- Fix: The C-1 transient storage lock provides in-contract defense independent of Authority configuration.

[M-3] Zero-Address Owner Not Guarded

- Location: `sc.sol:42` — constructor
- Solmate `Auth` accepts `address(0)` as owner; deployment error silently bricks the contract.
- Fix: `require(_owner != address(0), "zero owner");`

[M-4] `wantApprovalAmount` Unbounded vs `bulkWithdraw` Proceeds

- Location: `sc.sol:185-188`
- Queue dictates `wantApprovalAmount`; a sandwich attack on vault redemption price can cause `bulkWithdraw` proceeds to fall below `wantApprovalAmount`, reverting the solve.
- Fix: Assert `wantApprovalAmount <= assetsReceived` after `bulkWithdraw`.

LOW / INFO

- **[L-1]** `eETH/weETH` constants (`sc.sol:16-17`) — declared, never used, hardcode mainnet addresses. Remove.
 - **[L-2]** `AlreadyInSolveContext` error (`sc.sol:34`) — declared, never emitted. Was intended for the missing reentrancy/context guard. Wire it up or remove it.
 - **[I-1]** No token recovery — stranded tokens cannot be swept without a contract upgrade. Add a restricted `rescue(ERC20, uint256)`.
 - **[I-2]** Batch failure isolation depends on `AtomicQueue` — a single user reverting inside the queue may grief the entire batch. Verify `AtomicQueue` has per-user error isolation.
-

Contacting the White Hat

c0ffeebabe.eth has no public email or Twitter in their ENS records.

Best channel: Blockscan Chat

- URL: chat.blockscan.com
- Address: [0xC0ffeEBABE5D496B2DDE509f9fa189C25cF29671](#)
- Etherscan confirms they have active messages there

Message template:

I'm the owner of [0xE3FF7ccf609082BdA151F40790A1c9143B3a0663](#). You rescued ~1,203,824 HexTrustUSD in tx [INSERT TX HASH] via the AtomicSolverV3 finishSolve exploit. Please reach out to discuss return of funds. I can sign a message from the affected address to prove ownership.

Be prepared to sign a message from [0xE3FF7...](#) to prove ownership — standard white hat recovery procedure.

Recommended Immediate Actions

1. **Pause all AtomicSolverV3 operations** — revoke authorization for any remaining solvers until the contract is patched and redeployed
 2. **Audit the Authority contract** — determine which addresses currently hold [finishSolve](#) permission and revoke all until fix is deployed
 3. **Contact c0ffeebabe.eth** via Blockscan Chat (see above)
 4. **Deploy patched contract** with the C-1 transient storage fix, H-1 nonReentrant, and H-2 safeApprove zero-reset as minimum viable patch
 5. **Run Echidna/Medusa invariant fuzzing** on INV-2 before redeployment
 6. **Commission independent audit** from a qualified firm before handling significant value
-

Audit performed by AI agents (adversarial static analysis). Not a substitute for a professional security audit.